COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 - Data Structures and Algorithms | **SECTION:** | CPE21S3 |
| **DATE PERFORMED:** | September 17, 2026 | **DATE SUBMITTED:** | September 00, 2026 |
| **NAME:** | Garbosa, Isaiah Benedict R. | **INSTRUCTOR:** | Engr. Jimlord Quejado |

**Hands-on Activity 9.1 Tree ADT**

**1. PROCEDURE OUTPUT**

**ILO A: Create C++ code to implement a general tree, binary tree and binary search tree.**

**General Tree Code**

```
#ifndef GT_H
#define GT_H

template <typename T>
class GeneralTree{
    private:
        T key;
        GeneralTree<T>* firstChild;
        GeneralTree<T>* sibling;

    public:
        GeneralTree(T value);
        void insert(GeneralTree<T>* child);
        void preOrder();
};

template <typename T>
GeneralTree<T>::GeneralTree(T value){
    key = value;
    firstChild = nullptr;
    sibling = nullptr;
}
```

```cpp
template <typename T>
void GeneralTree<T>::insert(GeneralTree<T>* child){
    if(firstChild == nullptr){
        firstChild = child;
    }
    else{
        GeneralTree<T>* current = firstChild;
        while(current->sibling != nullptr){
            current = current->sibling;
        }
        current->sibling = child;
    }
}
```

```cpp
#include <iostream>
#include "GT.h"

int main(){

    GeneralTree<char>* A = new GeneralTree<char>('A');

    GeneralTree<char>* B = new GeneralTree<char>('B');
    GeneralTree<char>* C = new GeneralTree<char>('C');
    GeneralTree<char>* D = new GeneralTree<char>('D');
    GeneralTree<char>* E = new GeneralTree<char>('E');
    GeneralTree<char>* F = new GeneralTree<char>('F');
    GeneralTree<char>* G = new GeneralTree<char>('G');

    GeneralTree<char>* H = new GeneralTree<char>('H');
    GeneralTree<char>* I = new GeneralTree<char>('I');
    GeneralTree<char>* J = new GeneralTree<char>('J');

    GeneralTree<char>* K = new GeneralTree<char>('K');
    GeneralTree<char>* L = new GeneralTree<char>('L');
    GeneralTree<char>* M = new GeneralTree<char>('M');
    GeneralTree<char>* N = new GeneralTree<char>('N');

    GeneralTree<char>* P = new GeneralTree<char>('P');
    GeneralTree<char>* Q = new GeneralTree<char>('Q');
```

```
A->insert(B);
A->insert(C);
A->insert(D);
A->insert(E);
A->insert(F);
A->insert(G);

D->insert(H);

E->insert(I);
E->insert(J);

F->insert(K);
F->insert(L);
F->insert(M);

G->insert(N);

J->insert(P);
J->insert(Q);
```

| Observation/Comments |
| --- |
| For the general tree, there is no left or right. Instead, there is a child and a sibling. The program adds the node to the children if it does not have any children yet and will move through the sibling if there already is a child. The general tree is different in comparison to the binary tree as this tree doesn't only have a left and right node, it can have a node with multiple children and siblings. |

**Table 9.1. General Tree**

| Node | Height | Depth |
| --- | --- | --- |
| A | **3** | **0** |
| B | **0** | **1** |
| C | **0** | **1** |
| D | **1** | **1** |
| E | **2** | **1** |
| F | **1** | **1** |
| G | **1** | **1** |
| H | **0** | **2** |
| I | **0** | **2** |
| J | **1** | **2** |
| K | **0** | **2** |
| L | **0** | **2** |
| M | **0** | **2** |
| N | **0** | **2** |
| P | **0** | **3** |
| Q | **0** | **3** |

**Table 9.2. Height and Depth**

**ILO B: Create C++ code for implementation of tree traversal methods such as pre-order, in-order and postorder traversal.**

| | |
|---|---|
| **Pre-order** | **A -> B -> C -> D -> H -> E -> I -> J -> P -> Q -> F -> K -> L -> M -> G -> N** |
| **Post-order** | **B -> C -> D -> H -> I -> P -> Q -> J -> E -> K -> L -> M -> F -> N -> G -> A** |
| **In-order** | **B -> C -> D -> H -> A -> I -> E -> P -> J -> Q -> K -> F -> L -> M -> N -> G** |

**Table 9.3. Pre-order, Post-order, and In-order Traversal by hand**

| Code |
|---|
| **Pre-order** |

```cpp
template <typename T>
void GeneralTree<T>::preOrder(){

    std::cout << key << " ";

    if(firstChild != nullptr){
        firstChild->preOrder();
    }

    if(sibling != nullptr){
        sibling->preOrder();
    }
}
```

**Post-order**

```cpp
template <typename T>
void GeneralTree<T>::postOrder(){

    if(firstChild != nullptr){
        firstChild->postOrder();
    }

    if(sibling != nullptr){
        sibling->postOrder();
    }

    std::cout << key << " ";

}
```

**In-order**

```cpp
template <typename T>
void GeneralTree<T>::inOrder(){

    if(firstChild != nullptr){
        firstChild->inOrder();
    }

    std::cout << key << " ";

    if(sibling != nullptr){
        sibling->inOrder();
    }
}
```

**Output**

```
Pre order output: A B C D H E I J P Q F K L M G N
In order output: B C H D I P Q J E K L M F N G A
Post order output: H Q P J I M L K N G F E D C B A
```

**Observation**

Comparing the code output and the one done by hand, I can notice that I got the pre-order right but got the other two wrong. I observed that the sequence I did for post order was the sequence that was used in order by the program. The one I did for them in order was not present. I noticed that the post order was completely different. I believe that I got it wrong because I was using the logic for the binary tree where there is only a left and right for each node.

**Table 9.4. Pre-order, Post-order, and In-order traversal**

**Code**

```
template <typename T>
bool GeneralTree<T>::findPreOrder(T findData){

    if(key == findData){
        return true;
    }

    if(firstChild != nullptr){
        if(firstChild->findPreOrder(findData)){
            return true;
        }
    }

    if(sibling != nullptr){
        if(sibling->findPreOrder(findData)){
            return true;
        }
    }

    return false;
}
```

```cpp
template <typename T>
bool GeneralTree<T>::findInOrder(T findData){

    if(firstChild != nullptr){
        if(firstChild->findInOrder(findData)){
            return true;
        }
    }

    if(key == findData){
        return true;
    }

    if(sibling != nullptr){
        if(sibling->findInOrder(findData)){
            return true;
        }
    }
    return false;
}
```

```cpp
template <typename T>
bool GeneralTree<T>::findPostOrder(T findData){

    if(firstChild != nullptr){
        if(firstChild->findPostOrder(findData)){
            return true;
        }
    }
    if(sibling != nullptr){
        if(sibling->findPostOrder(findData)){
            return true;
        }
    }

    if(key == findData){
        return true;
    }

    return false;
}
```

```cpp
template <typename T>
void GeneralTree<T>::findData(int choice, T key){
    bool found = false;

    if(choice == 1){
        found = findPreOrder(key);
    }

    else if(choice == 2){
        found = findInOrder(key);
    }
    else if(choice == 3){
        found = findPostOrder(key);
    }
    if(found){
        std::cout<<key<<" was found"<<std::endl;
    }
}
```

```cpp
A->findData(1,'E');
A->findData(2,'D');
A->findData(3,'F');
A->findData(1,'W');
```

**Output**

```
E was found
D was found
F was found
```

**Table 9.5. Find Data**

**Code**

```
G->insert(O);
```

```
GeneralTree<char>* O = new GeneralTree<char>('O');
```

```
A->findData(1,'O');
A->findData(2,'O');
A->findData(3,'O');
```

**Output**

```
O was found
O was found
O was found
```

| Observation |
| --- |
| The new leaf “O” which was the child of node G was successfully added to the tree. By using the findData function, we can find out that it is added successfully. |

**Table 9.6. Adding New Leaf**

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

**ILO C: Solve given problems using the tree data structure’s implementation in C++**

**Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.**

```
int main(){

    BinarySearchTree<int>* tree = new BinarySearchTree(2);

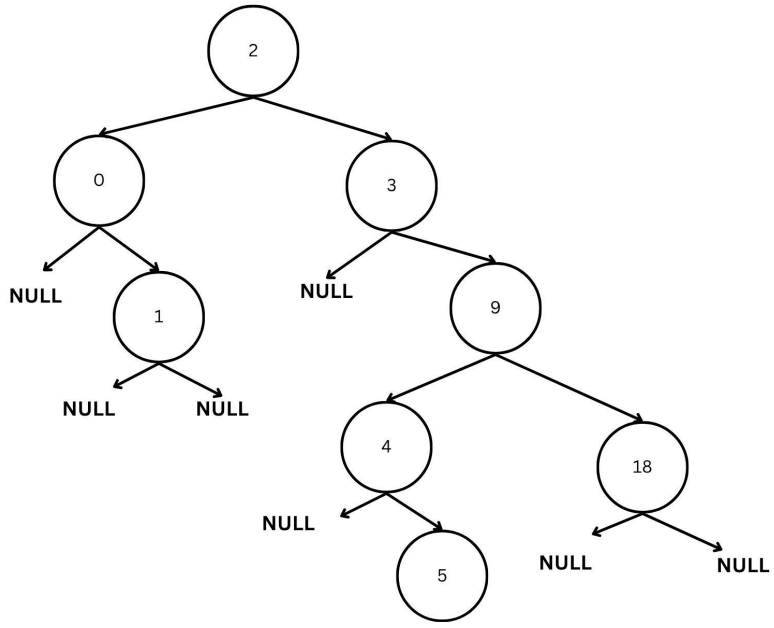
    tree->insert(3);
    tree->insert(9);
    tree->insert(18);
    tree->insert(0);
    tree->insert(1);
    tree->insert(4);
    tree->insert(5);
```

```cpp
template <typename T>
BinarySearchTree<T>::BinarySearchTree(T value){
    key = value;
    left = nullptr;
    right = nullptr;
}

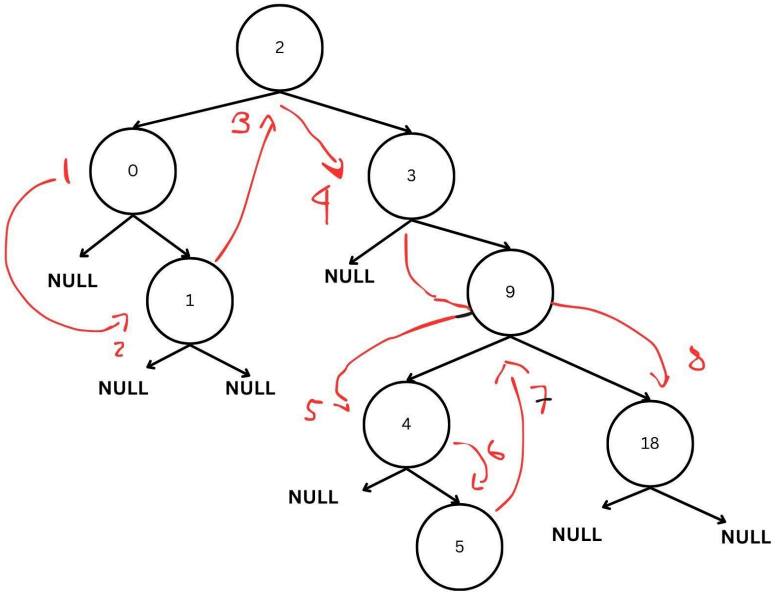
template <typename T>
void BinarySearchTree<T>::insert(T value){
    //go to left if value < key
    if(value < key){

        if(left == nullptr){
            left = new BinarySearchTree(value);
        }
        else{
            left->insert(value);
        }
    }
    else if(value > key){
        if(right == nullptr){
            right = new BinarySearchTree(value);
        }
        else{
            right -> insert(value);
        }
    }
}
```

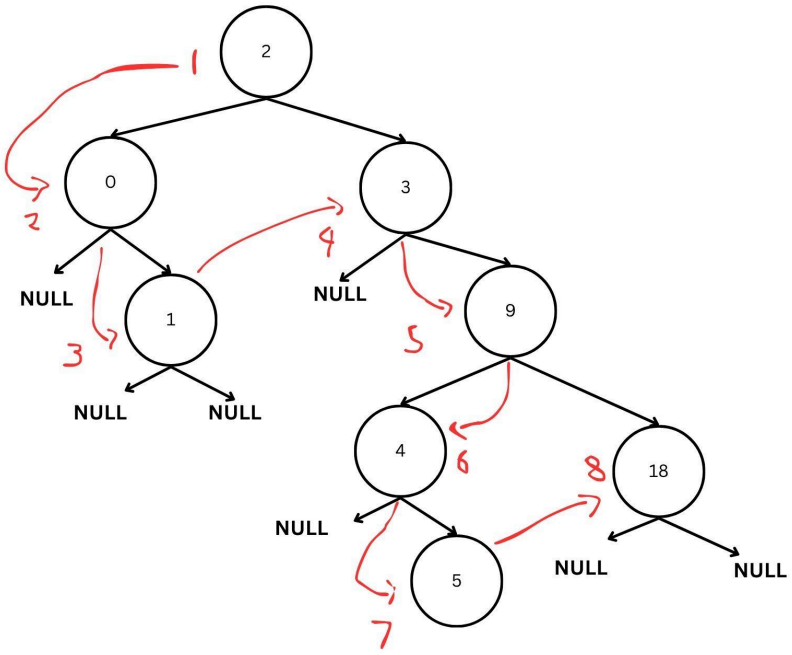
**Step 2: Create a diagram to show the tree after all values have been inserted.**



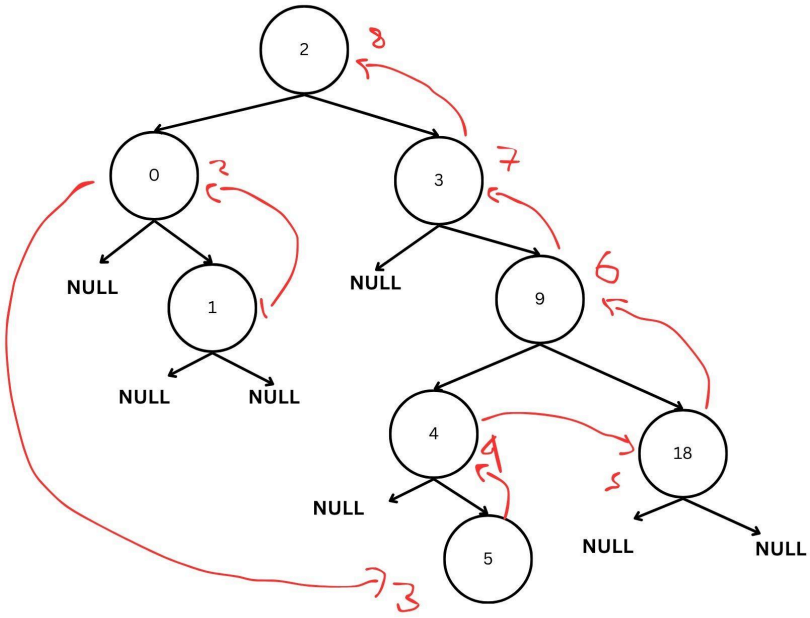

**Traversal order for in-order**



## Traversal order for pre-order




## Traversal order for post-order



**Step 3: Compare the different traversal methods. In-order traversal was performed with what function?**

**Code (In-order)**

```
template <typename T>
void BinarySearchTree<T>::inOrder(){

    if(left != nullptr){
        left->inOrder();
    }

    std::cout<<key<<" ";

    if(right != nullptr){
        right->inOrder();
    }
}
```

**Output (In-order)**

```
In order output: 0 1 2 3 4 5 9 18
```

**Code (Pre-order)**

```
template <typename T>
void BinarySearchTree<T>::preOrder(){
    std::cout<<key<<" ";

    if(left != nullptr){
        left -> preOrder();
    }

    if(right != nullptr){
        right->preOrder();
    }
}
```

**Output (Pre-order)**




**Code (Post-order)**

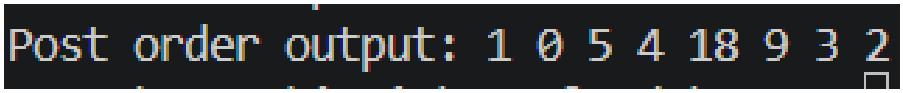
```
template <typename T>
void BinarySearchTree<T>::postOrder(){
    if(left != nullptr){
        left->postOrder();
    }

    if(right != nullptr){
        right->postOrder();
    }

    std::cout<<key<<" ";

}
```

**Output (Post-order)**




**Explanation**

For these codes, I implemented pre-order, in-order, and post-order traversal. The Pre-order traversal was implemented by starting with the root, then checking if the left and right is a nullptr. The left and right nodes will also call the function. The In-order traversal starts by navigating to the very left of the tree. It then goes to the root and the right subtree. For the post-order, it navigates to the left and moves to the right subtree, making its way up to the root.

| Conclusion |
| --- |
| The activity focused on how to implement a binary tree and general tree, as well as the different traversal methods that can be used to display the leaves of the tree. There are 3 different traversal methods that can be applied. First is the pre-order which starts from the root, then traverses to the left, and to the right. The in-order traversal starts from the very left, visits the root, and goes to the right subtree. The post-order traversal traverses to the left and right subtrees first before making it to the root. Overall, I was able to learn the basic functionality and how trees work in data structure. However, I still find myself struggling to keep up and process the different concepts that was in this activity and thus, I will try to learn and study more about trees. |

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **N/A**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **N/A**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **N/A**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.